

Lecture 12: Getting Started & Core Concepts

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhlai

February 4, 2026

Agenda

1. The Spark Entry Point
2. RDDs: Resilient Distributed Datasets
3. Spark Operations
4. Lazy Evaluation & DAG
5. Shared Variables
6. Persistence (Caching)
7. Summary

The Spark Entry Point

The Entry Point

- Every Spark application needs an entry point to communicate with the cluster.
- In Spark 1.x, we had multiple contexts:
 - `SparkContext`: Main entry point for core Spark functionality.
 - `SQLContext`: For working with Structured data.
 - `HiveContext`: For Hive integration.
- In Spark 2.x+, these were unified into `SparkSession`.

SparkSession: The Unified Entry Point

SparkSession encapsulates the functionality of SparkContext and SQLContext.

- It allows you to create DataFrames, register tables, execute SQL, and access the underlying SparkContext.

```
1 import org.apache.spark.sql.SparkSession
2
3 val spark = SparkSession.builder()
4     .appName("My Spark App")
5     .master("local[*]") // Run locally using all cores
6     .getOrCreate()
7
8 // Accessing the underlying SparkContext if needed
9 val sc = spark.sparkContext
```

Listing 1: Creating a SparkSession in Scala

RDDs: Resilient Distributed Datasets

What is an RDD?

Resilient Distributed Dataset

- The fundamental data structure of Spark.
- Represents a collection of elements partitioned across the nodes of the cluster that can be operated on in parallel.

Key Characteristics:

- **Resilient:** Fault-tolerant (recomputes missing partitions using lineage).
- **Distributed:** Data resides on multiple nodes.
- **Dataset:** Collection of partitioned data.

Properties of an RDD

Internally, each RDD is characterized by five main properties:

1. A list of **partitions**.
2. A function for computing each split.
3. A list of **dependencies** on other RDDs.
4. Optionally, a **Partitioner** for key-value RDDs (e.g., HashPartitioner).
5. Optionally, a list of **preferred locations** (data locality).

Creating RDDs

1. Parallelizing an existing collection:

```
1 val data = Seq(1, 2, 3, 4, 5)
2 val rdd1 = spark.sparkContext.parallelize(data)
```

2. Loading from external datasets:

```
1 // Reads text file; each line becomes an element
2 val rdd2 = spark.sparkContext.textFile("hdfs:///path/to/file.txt")
```

Spark Operations

Two Types of Operations

Spark operations are classified into two categories:

1. Transformations

- Create a new RDD from an existing one.
- **Lazy:** Not computed immediately.
- Example: `map`, `filter`, `groupByKey`.

2. Actions

- Return a value to the driver program or write data to storage.
- Triggers the actual computation.
- Example: `count`, `collect`, `saveAsTextFile`.

Transformations: Narrow vs. Wide

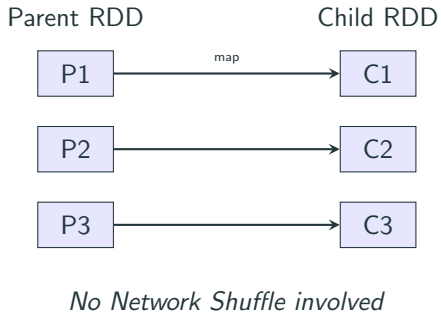
- **Narrow Transformation:**

- Each partition of the parent RDD is used by at most one partition of the child RDD.
- **No Shuffle** required. Fast.
- E.g., `map`, `filter`.

- **Wide Transformation:**

- Multiple child partitions may depend on a single parent partition.
- **Shuffle** required (data exchange across network). Expensive.
- E.g., `groupByKey`, `reduceByKey`.

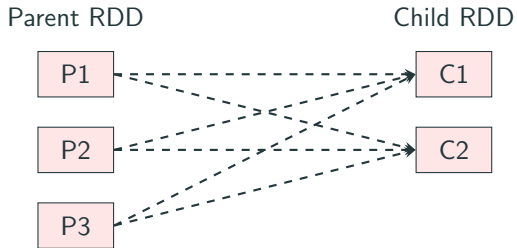
Visualizing Narrow Transformations



Narrow Transformation Examples

```
1  val numbers = sc.parallelize(1 to 10)
2
3  // Map: x => x * 2
4  val doubled = numbers.map(_ * 2)
5
6  // Filter: x => x % 2 == 0
7  val evens = numbers.filter(_ % 2 == 0)
8
9  // FlatMap: One input -> Many outputs
10 val lines = sc.parallelize(List("Hello World", "Apache Spark"))
11 val words = lines.flatMap(line => line.split(" "))
12 // Result: "Hello", "World", "Apache", "Spark"
```

Visualizing Wide Transformations (Shuffle)



Shuffle: Data redistributed based on Keys

Wide Transformation Examples

```
1 val pairs = sc.parallelize(List(  
2     ("a", 1), ("b", 2), ("a", 3), ("b", 4)  
3 ))  
4  
5 // GroupByKey: Groups values by key  
6 // ("a", [1, 3]), ("b", [2, 4])  
7 val grouped = pairs.groupByKey()  
8  
9 // ReduceByKey: Aggregates values by key locally then globally  
10 // ("a", 4), ("b", 6)  
11 val counts = pairs.reduceByKey((acc, value) => acc + value)
```

Note: reduceByKey is preferred over groupByKey as it performs map-side combination (reducing data transfer).

Actions: Triggering Computation

```
1  val rdd = sc.parallelize(1 to 100)
2  val filtered = rdd.filter(_ > 50)  // Nothing happens here!
3
4  // Action 1: Count elements
5  val count = filtered.count()        // Computation starts here
6  println(count)
7
8  // Action 2: Collect results to Driver
9  // WARNING: Don't use on large datasets! OOM Risk.
10 val result = filtered.collect()
11
12 // Action 3: Save to disk
13 filtered.saveAsTextFile("output/dir")
```

Lazy Evaluation & DAG

Lazy Evaluation

- Spark does not evaluate transformations immediately.
- Instead, it builds a **Logical Plan** (recipe) of transformations.
- Execution is only triggered when an **Action** is called.

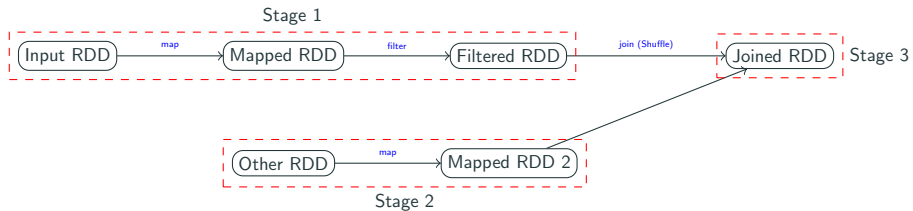
Benefits:

- **Optimization:** Spark can optimize the entire chain of operations (e.g., combine multiple maps, filter early).
- **Fault Tolerance:** If a node fails, Spark knows exactly how to recompute the lost data using the lineage.

Lineage Graph (DAG)

- **DAG:** Directed Acyclic Graph.
- Nodes = RDDs (or data blocks).
- Edges = Transformations.
- When an Action is called, the DAG is submitted to the DAG Scheduler.

Visualizing the DAG



Jobs, Stages, and Tasks

- **Job:** Created when an Action is triggered.
- **Stage:** Boundary created by a **Shuffle** (Wide Transformation).
 - Inside a stage, tasks can be pipelined (narrow transformations).
- **Task:** The smallest unit of work. Sent to one executor.
 - One task per partition.

Action → Job → Stages → Tasks

Shared Variables

The Closure Problem

When code is executed on a cluster:

- Spark breaks up the processing of RDD operations into tasks.
- Each task is executed by an "Executor" process.
- Variables used in the RDD operations are **copied** to each task.
- Updates to variables on executors **are not propagated back** to the driver!

The Problem Illustrated

```
1 var counter = 0
2 val rdd = sc.parallelize(data)
3
4 // This executes on executors
5 rdd.foreach(x => counter += x)
6
7 println("Counter value: " + counter)
8 // Output: Counter value: 0
9 // Because the 'counter' on driver was never updated!
```

Solution: Shared Variables

Spark provides two types of shared variables for common usage patterns:

1. **Broadcast Variables:** Efficiently read-only shared data.
2. **Accumulators:** Variables that are only "added" to (associative and commutative).

Broadcast Variables

- Keep a read-only variable cached on each **machine** (executor) rather than shipping a copy of it with **tasks**.
- Reduces network overhead for large reference data (e.g., a lookup table or ML model).
- Uses efficient P2P protocols (BitTorrent-like) to distribute data.

Broadcast Variable Example

```
1 // A large lookup table
2 val lookup = Map(1 -> "a", 2 -> "b", 3 -> "c")
3
4 // Broadcast it
5 val broadcastLookup = sc.broadcast(lookup)
6
7 val data = sc.parallelize(Seq(1, 2, 3))
8
9 val result = data.map(id => {
10     // Access via .value
11     broadcastLookup.value.getOrElse(id, "unknown")
12 }).collect()
```

Accumulators

- Used to aggregate information from workers back to the driver.
- Only the driver can read the value.
- Executors can only add to it.
- Common uses: Counters, Sums, Debugging.

Accumulator Example

```
1  val accum = sc.longAccumulator("My Accumulator")
2
3  sc.parallelize(Array(1, 2, 3, 4)).foreach(x => {
4      accum.add(x)
5  })
6
7  println(accum.value) // Output: 10
```

Persistence (Caching)

Why Cache?

- By default, each transformed RDD is recomputed each time you run an action on it.
- If you plan to reuse an RDD (e.g., in iterative algorithms like PageRank or machine learning), **persist** it.
- Persistence saves the RDD elements in memory (or disk) on the executors.

Cache vs. Persist

- `cache()`: Default persistence. Stores as `MEMORY_ONLY`.
- `persist(storageLevel)`: Allows customization.

```
1 import org.apache.spark.storage.StorageLevel
2
3 val rdd = sc.textFile("../").map(...)
4
5 // Cache in memory
6 rdd.cache()
7
8 // Persist with specific level (e.g., Memory + Disk)
9 rdd.persist(StorageLevel.MEMORY_AND_DISK)
```

Storage Levels

Storage Level	Description
MEMORY_ONLY	Store in RAM. Resilient (recompute if lost). Fast.
MEMORY_AND_DISK	RAM first. Spill to disk if full.
DISK_ONLY	Store only on disk. Slow.
MEMORY_ONLY_SER	Store as serialized byte array. Space efficient.

Removing from Cache

- Spark automatically monitors cache usage and drops old data (LRU - Least Recently Used) if memory is full.
- You can manually remove an RDD using:

```
rdd.unpersist()
```

Summary

Recap

1. **SparkSession:** Unified entry point.
2. **RDDs:** Immutable, Partitioned, Resilient.
3. **Transformations:** Lazy, build the DAG (Narrow vs. Wide).
4. **Actions:** Trigger the job execution.
5. **Shared Variables:**
 - Broadcast (Read-only distribution).
 - Accumulators (Write-only aggregation).
6. **Persistence:** Crucial for performance in iterative jobs.

Questions?